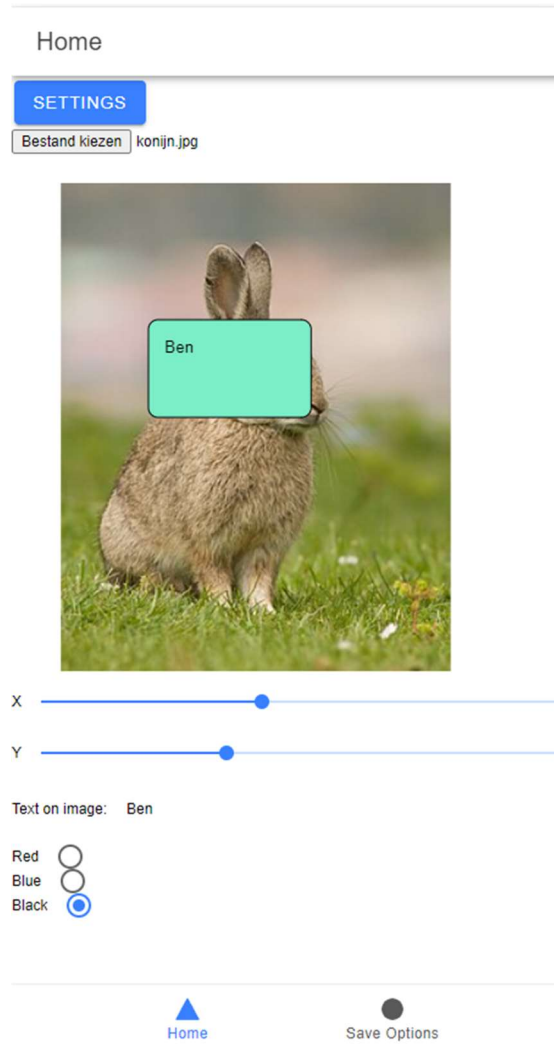


Cross Development – Ionic

Ben Van Damme r0890051



Inleiding.

Tijdens deze opdracht gebruikte we het Ionic framework om onze test case applicatie te bouwen. Ionic word gebruikt om hybride mobiele applicaties te maken. Kan ook gebruikt worden in desktop en progressive web apps. De originele versie die uitkwam in 2013 was gebouwt op Angular en Cordova. Op een moment is het opnieuw opgebouwt geweest als een set van web componenten. Daardoor kan je nu uit meer verschillende frameworks kan kiezen zoals Angular, React of Vue.js. Het is ook mogelijk om zonder deze eraan te beginnen doormiddel van de Ionic componenten. Mobiele applicaties kunne dan ook verkregen worden in native app stores en worden op (mobiele) apparaten geïnstalleerd doormiddel van Cordova of Capacitor.

Wat is een hybride app?

Hybride apps zijn applicaties die heel hard lijken op native apps. Je kan ze downloaden in de Apple App Store of Google Play Store. Het grote verschil is dat je hybride app direct werkt op meerdere platformen. Je hoeft dus geen aparte app per platform te maken.

Wat zijn de voordelen van een hybride app?

Het grootste voordeel van een hybride app tegenover een native app is de kost. Doordat je maar 1 app maakt die op verschillende platformen werkt in plaats van op elk platform een aparte app te ontwikkelen bespaar je ontwikkel uren en dus geld. De laatste jaren is de techniek verder ontwikkeld waardoor het steeds gebruiksvriendelijker word. Soms valt het verschil amper te merken of het een native of hybride app is.

Capacitor.

Capacitor is een alternatief op Cordova. Het is een nieuwe open source hybride app runtime gemaakt door het team van Ionic. Capacitor runt web applicaties natively op iOS, Android en Web. Hij heeft dezelfde cross-platform voordelen maar dan met een modernere aanpak op vlak van applicatie development en benutten de voordelen uit de laatste web API's en mogelijkheden die een native platform met zich meebrengt. Bijvoorbeeld bij het aanspreken van de camera word dezelfde code gebruikt voor zowel android als iOS, Electron of het web. Capacitor apps zijn eigenlijk bijna native apps. Ze gebruiken de native controls en kunnen aan elke native SDK of API van het platform waarop ze gebruikt worden. Ze worden in een embedded WebView gerunt en zo haal je dus de voordelen uit cross-platform.

Ionic vs Qt

Ionic en Qt zijn 2 verschillende frameworks die elks eigen doeleinden hebben

1. Ionic:
 - a. Type: Ionic is een framework om cross-platform mobiele applicaties te bouwen.
 - b. Taal: De talen die gebruikt worden zijn talen voor het web zoals HTML, CSS en javascript/typescript
 - c. Gebruik: Ionic word meestal gebruikt voor het ontwikkelen van mobiele apps die op verschillende platformen runnen zoals iOS, Android en web. Zo word er eigenlijk 1 keer code geschreven en dit werkt dan op verschillende platformen. Het Ionic framework komt met een set van libraries en tools voor het maken van web applicaties met de native look en feel op de verschillende platformen.
2. Qt:
 - a. Type: Qt is een cross-platform applicatie framework.
 - b. Taal: Qt word gebruikt met C++ als de hoofdtaal. Maar er worden ook opties aangeboden om aan de slag te kunne gaan met bijvoorbeeld Python.
 - c. Gebruik: Qt kan gebruikt worden voor veel verschillende soorten applicaties zoals desktop applicaties, mobiele applicaties en voor embedded systemen, en zelfs web applicaties. Ook hier word er gegaan voor de native look en feel op deze platformen.
3. Vergelijking tussen Ionic en Qt:
 - a. Scope:
 - i. Ionic is vooral gericht op mobiele applicaties en werkt heel goed als het word gebruikt voor het maken van apps met een consistente user interface op verschillende platformen.
 - ii. Qt is een alomvattend framework dat gebruikt word voor een groter aantal doeleinden als Ionic. Zo word het ook gebruikt voor desktop applicaties en embedded systemen.
 - b. Taal:
 - i. Ionic steunt op web technologieën. Zo word het toegankelijk voor web developers die HTML, CSS, Javascript en Typescript gebruiken.

- ii. Qt gebruikt vooral c++ ook is het mogelijk doormiddel van “bindings” om andere talen te gebruiken.
- c. Platform ondersteuning:
 - i. Ionic focust zich vooral op mobiele platformen zoals Android iOS. Het kan ook gebruikt worden om progressive webapps te maken(PWAs)
 - ii. Qt ondersteunt vele platformen zoals desktop operating systemen, embedded systemen en mobiele platformen.
- d. Prestaties
 - i. Qt is een native applicatie framework, waardoor de prestaties beter zijn omdat het compileerd naar machine code. Dit kan een voordeel zijn in scenarios waarbij prestaties belangrijker zijn.
 - ii. Ionic applicaties leveren niet altijd dezelfde prestaties als native apps. Maar doordat er veel vooruitgang wordt geboekt in browser technologie en hardware van mobiele apparaten verbeteren ook de prestaties van Ionic apps.
- e. Use Cases:
 - i. Ionic werkt goed voor projecten waar een eenzelfde code wordt gebruikt op verschillende mobiele platformen.
 - ii. Qt is goed voor een bredere waaier aan projecten zoals project met native performance, uitgebreide user interfaces of implementaties op verschillende platformen die verder reiken dan mobiel.

Mijn test applicatie :

Met mijn test applicatie kan de gebruiker een afbeelding inladen. Daarna kan hij in een tekstballon tekst schrijven. Deze tekstballon kan je veranderen van locatie. Ook kan je de kiezen welke kleur de omranding heeft van de tekstballon. Als je tevreden bent over het resultaat kan je naar de “Save Options” tab gaan en een naam ingeven en dan opslagen.

Technische beschrijving van de applicatie.

Om 2 pagina's te hebben gebruik ik tabs. 1 tab voor de main functionaliteit zoals het bewerken van de afbeelding en een andere tab om de bewerkte afbeelding op te slaan.

Om een afbeelding op te halen gebruik ik de `ion-input` tag. En om hem vervolgens te laten zien op de pagina gebruik ik een canvas met de `canvas` tag. Als er een bestand wordt geselecteerd in de `ion-input` wordt de `choosefile` functie opgeroepen. Deze maakt een `FileReader` aan. Vervolgens wordt het pad van dit bestand doorgestuurd naar mijn “CanvasService”. In deze canvas service gebeurt het volgende:

```

import { Injectable, ElementRef, EventEmitter } from '@angular/core';

@Injectable({
  providedIn: 'root'
})
export class CanvasService {
  sourceFile!: File;
  buildString: String = '';
  url: any;
  imageLoaded: boolean = false;

  sliderX = 0;
  sliderY = 0;
  text = '';
  outlineColor = "#000000";

  drawnOnMain: boolean = false;
  canvas!: ElementRef;
  context!: any;
  Drawn: EventEmitter<any> = new EventEmitter();
  constructor() { }

  async drawImageOnCanvas(): Promise<number> {
    return new Promise((resolve, reject) => {

      const img = new Image();
      img.src = this.url;
      img.onload = () => {
        const canvasWidth = this.canvas.nativeElement.width;
        const canvasHeight = this.canvas.nativeElement.height;

        const aspectRatio = img.width / img.height;
        let drawWidth = canvasWidth;
        let drawHeight = canvasWidth / aspectRatio;

        if (drawHeight > canvasHeight) {
          drawHeight = canvasHeight;
          drawWidth = canvasHeight * aspectRatio;
        }

        const x = (canvasWidth - drawWidth) / 2;
        const y = (canvasHeight - drawHeight) / 2;

        this.context.clearRect(0, 0, canvasWidth, canvasHeight);

        this.context.drawImage(img, x, y, drawWidth, drawHeight);
        this.drawRectangleOnCanvas();
        console.log("text is ", this.text);
        this.drawText();
        this.imageLoaded = true;
      }

      setTimeout(() => {
        const randomNumber = 1;
        resolve(randomNumber);
      }, 500);
    });
  }
}

```

```

drawRectangleOnCanvas(): void{
  this.context.fillStyle = "#7bedc7";
  const canvasWidth = this.canvas.nativeElement.width;
  const canvasHeight = this.canvas.nativeElement.height;
  const rectWidth = this.canvas.nativeElement.width/3;
  const rectHeight = this.canvas.nativeElement.height/5;
  const radius = canvasHeight/50;

  this.roundedRect(this.context, (((canvasWidth-rectWidth)/100)*this.sliderX, ((canvas
}

private roundedRect(ctx: CanvasRenderingContext2D, x: number, y: number, width: number, height: number, radius: number){
  ctx.beginPath();
  ctx.moveTo(x + radius, y);
  ctx.arcTo(x + width, y, x + width, y + height, radius);
  ctx.arcTo(x + width, y + height, x, y + height, radius);
  ctx.arcTo(x, y + height, x, y, radius);
  ctx.arcTo(x, y, x + width, y, radius);
  ctx.closePath();
  ctx.fill();
  ctx.lineWidth = 1;
  ctx.strokeStyle = this.outlineColor;
  ctx.stroke();
}

drawText():void{
  if (this.context) {
    const fontSize = this.canvas.nativeElement.width/30;
    this.context.font = fontSize.toString()+'px Arial';
    this.context.fillStyle = this.outlineColor;
    const canvasWidth = this.canvas.nativeElement.width;
    const canvasHeight = this.canvas.nativeElement.height;
    const rectWidth = this.canvas.nativeElement.width/3;
    const rectHeight = this.canvas.nativeElement.height/5;
    const textmarginx = this.canvas.nativeElement.width/30;
    const textmarginx = this.canvas.nativeElement.width/15;

    this.context.fillText(this.text, (((((canvasWidth-rectWidth)/100)*this.sliderX)+textmarginx), ((canvasHeight-rectHeight)/100)*this.sliderY);
  }
}
}

```

De functie “drawimageoncanvas” word opgeroepen. Het is een asynchrone functie omdat het even duurt om grafische bewerkingen te doen. Ik bereken de hoogte en breedte van het canvas en plaats dan de afbeelding in het canvas met het correcte aspect ratio zodat de afbeelding niet uit proportie word getrokken. Na dat de afbeelding op het canvas is getekent ga ik een rounded rechthoek tekenen op het canvas. Dit op de coördinaten die ingesteld staan door mijn sliders.

Nadien kan ik tekst op de rechthoek tekenen deze tekst kan de gebruiker zelf ingeven. Als de tekst verandert of de waarden van de slider zullen alle draw functies terug worden uitgevoerd om de verandering te kunnen laten zien.

Beneden het canvas hebben we de sliders. 1 voor de X coördinaat en 1 voor de y coördinaat. Als een slider gewijzigd word dan worden de teken functies terug oepgroeppen.

Onder de slider sectie komt het textvak waar je de tekst kan ingeven die je in het tekstballon terug wilt zien. Je typt een zin en je duwt op enter en dan verschijnt de tekst in het vak.

Vervolgens komen de radiobuttons waar je kiest welke kleur de tekst en de rand van de tekstballon moeten hebben.

Als je tevreden bent met je resultaat kan je onderaan klikken op de “save options” tab. Een andere pagina opent zichzelf waar zich een textvak bevind voor de naam van het bestand in te geven dat je wilt opslagen en een save knop. Als je na het ingeven van een bestandsnaam op save duwt word de bewerkte afbeelding opgeslagen onder de documents folder op je smartphone. Hiervoor heb ik een plugin gebruikt genaamt `@capacitor/filesystem`. En deze installeer je door in je Command Line Interface naar je project te gaan en dan deze command uitvoeren: `npm install @capacitor/filesystem` Ik gebruik deze plugin om een afbeelding te kunnen wegschrijven naar de smartphone zijn filesystem.

@capacitor/filesystem

The Filesystem API provides a NodeJS-like API for working with files on the device.

Install

```
npm install @capacitor/filesystem
npx cap sync
```

De bewerkte afbeelding dat zich op het canvas bevind omzette naar een writable bestand doe ik als volgt: eerst maak ik van het canvas een dataURL vervolgens maak ik er een blob van en daarna word het omgezet in base64 data. Dan schrijf ik het bestand weg de Filesystem plugin.

Save Options

File Name

SAVE

preview:



▲
Home

●
Save Options

```

saveIt():void{
  var dataUrl = this.canvasService.canvas.nativeElement.toDataURL();
  const blob = this.dataURLtoBlob(dataUrl);
  this.writeFile(blob);
}

private dataURLtoBlob(dataURL: string): Blob {
  const arr = dataURL.split(',');
  const matchResult = arr[0].match(/:(.*?)/);

  if (matchResult && matchResult[1]) {
    const mime = matchResult[1];
    const bstr = atob(arr[1]);
    let n = bstr.length;
    const u8arr = new Uint8Array(n);

    while (n--) {
      u8arr[n] = bstr.charCodeAt(n);
    }

    return new Blob([u8arr], { type: mime });
  } else {
    throw new Error('Invalid data URL format');
  }
}

async writeFile(bl: Blob) {
  const base64Data = await this.blobToBase64(bl);

  await Filesystem.writeFile({
    path: `${this.filename}.jpeg`,
    data: base64Data,
    directory: Directory.Documents,
  });
}

private async blobToBase64(blob: Blob): Promise<string> {
  return new Promise((resolve, reject) => {
    const reader = new FileReader();
    reader.onloadend = () => {
      if (typeof reader.result === 'string') {
        resolve(reader.result);
      } else {
        reject(new Error('Failed to convert Blob to base64'));
      }
    };
    reader.readAsDataURL(blob);
  });
}
}

```

Onder de save knop bevindt zich de preview die word geupdate als de afbeelding in de home tab word aangepast. Doormiddel van een event te gebruiken weet de preview da hij zich opnieuw moet aanpassen.

Tenslotte zijn er nog de settings die zich bevinden op de home page als je op de knop settings duwt. Komt er alert op je scherm. Dit is een ion-alert. Je kiest eerst welke setting je wilt aanpassen je kan kiezen uit Font Size of Font Color en duwt vervolgens op "OK". Nadien krijg je verder opties die te maken hebben met de optie die je vooraf hebt gekozen.

De applicatie runnen op een Android smartphone.

Om de applicatie de laten werken op android moet je eerst de applicatie permissie geven om te kunnen schrijven en lezen van en naar external storage.

Dit kan je aanpassen in de android manifest file.

Android

If using `Directory.Documents` or `Directory.ExternalStorage`, in Android 10 and older, this API requires the following permissions be added to your `AndroidManifest.xml`:

```
<uses-permission android:name="android.permission.READ_EXTERNAL_STORAGE"/>
<uses-permission android:name="android.permission.WRITE_EXTERNAL_STORAGE" />
```

File Edit View Navigate Code Refactor Build Run Tools Git Window Help

android - AndroidManifest.xml [android.app.main]

Project

Android

app

manifests

AndroidManifest.xml

java

assets

public

capacitor.config.json

capacitor.plugins.json

res

res (generated)

capacitor-android

capacitor-app

manifests

AndroidManifest.xml

java

com.capacitorjs.plugins.app

AppPlugin

java (generated)

com.capacitorjs.plugins.app

build.config

res (generated)

capacitor-cordova-android-plugins

manifests

AndroidManifest.xml

java

org.apache.cordova.build.core

plugin

Logcat

Logcat

+

Pixel 6 Pro API 24.2 Android 0, API 24 [OFFLINE]

package:mine

capacitor.config.json

capacitor.plugins.json

AndroidManifest.xml

capacitor.config.json

</activity>

<provider

android:name="androidx.core.content.FileProvider"

android:authorities="\${applicationId}.fileprovider"

android:exported="false"

android:grantUriPermissions="true">

<meta-data

android:name="android.support.FILE_PROVIDER_PATHS"

android:resource="@xml/file_paths"></meta-data>

</provider>

</application>

<!-- Permissions -->

<uses-permission android:name="android.permission.WRITE_EXTERNAL_STORAGE" />

<uses-permission android:name="android.permission.INTERNET" />

</manifest>

manifest

uses-permission

Merged Manifest

Device Manager

Virtual

Physical

Create device

?

Device

Pixel 4 API 33

Android 13.0 | 685

33

8.9 GB

...

Pixel 6 Pro API 24

Android 7.0 Google APIs | 685

24

9.7 GB

...

Pixel 6 Pro API 24.2

Android 7.0 Google APIs | 685

24

9.2 GB

...

Checking What's New Assistant version...

39:30 2 spaces

FRA

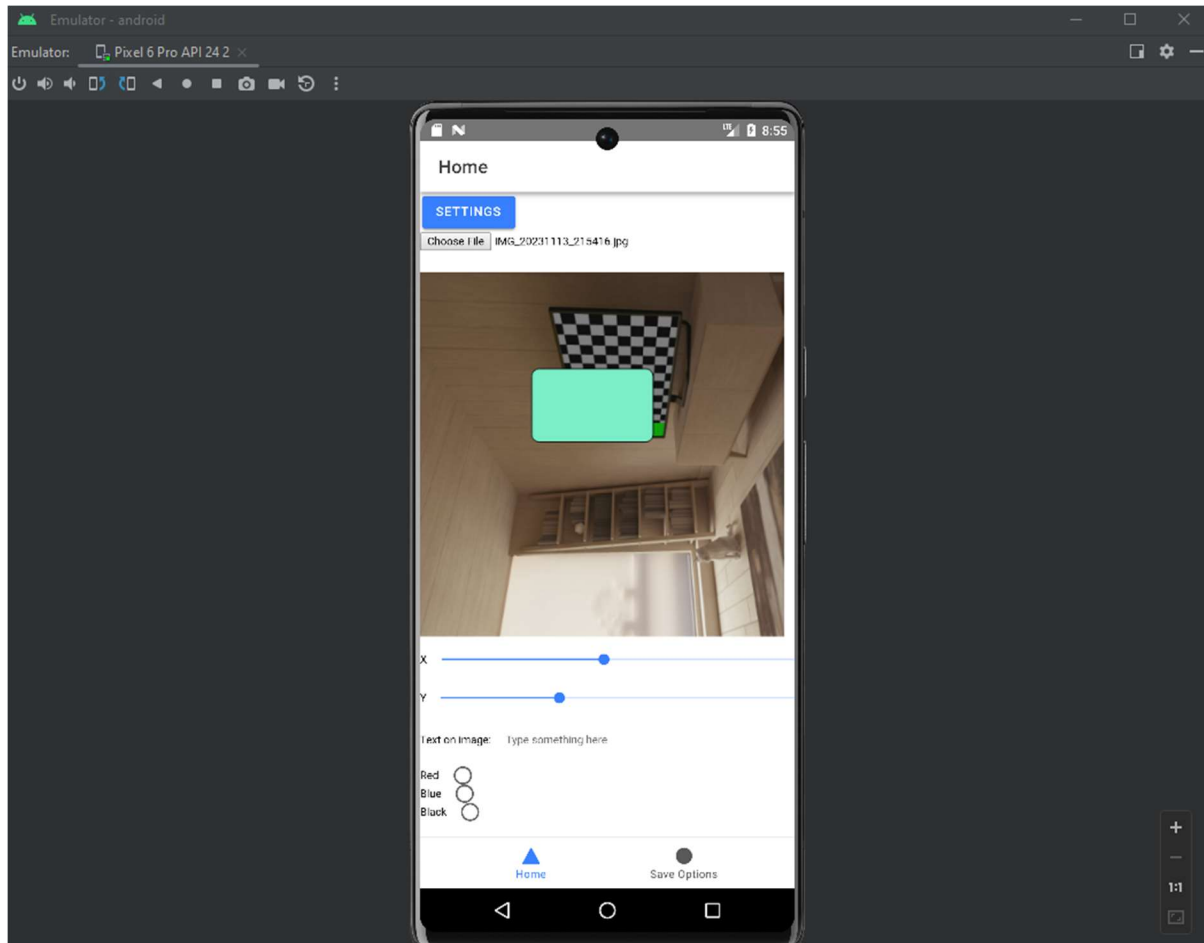
24/11/2023

Vervolgen moet je in je CLI het path naar je java installatie invoeren elke keer als ik mijn CLI heropen: `set JAVA_HOME=C:\Program Files\Eclipse Adoptium\jdk-17.0.6.10-hotspot`

Daarna start je android studio op en start je een emulator.

Met het volgende commando start je te app op je emulator: `ionic cap run android --prod`

Nadat je hebt gekozen welk apparaat je wilt gebruiken word de app gelaunched en krijg je het volgende te zien



Bronnen:

[https://en.wikipedia.org/wiki/Ionic_\(mobile_app_framework\)#:~:text=Ionic%20uses%20Cordova%20and%2C%20more,Electron\)%2C%20or%20modern%20browsers.](https://en.wikipedia.org/wiki/Ionic_(mobile_app_framework)#:~:text=Ionic%20uses%20Cordova%20and%2C%20more,Electron)%2C%20or%20modern%20browsers.)

<https://ionic.io/resources/articles/capacitor-vs-cordova-modern-hybrid-app-development>

<https://coffeeit.nl/verschil-tussen-hybride-native-en-web-apps/#:~:text=Hybride%20apps%20zijn%20applicaties%20die,twee%20aparte%20apps%20te%20bouwen.>

Chat.openai.com

Mijn project code

De powerpoints van de les.